

עיבוד אותות - פרויקט 1

[קישור למחברת בקולאב](#)

חלק 1: משפט הדגימה

ראשית, התחלתי מסידור הדאטה וחלוקה של האות למקטעים שונים, על ידי שימוש ב-mask:

```
# Prepping the system and data.
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from scipy import signal

path_neural = '/content/drive/MyDrive/Signal Processing/Project #1/Neural_data.csv'
path_spikes = '/content/drive/MyDrive/Signal Processing/Project #1/spike_trains.csv'
df_neural = pd.read_csv(path_neural)
df_spikes = pd.read_csv(path_spikes)
```

```
# Extract the neural signal and define the time vector.
data = df_neural['ΔF/F (%)'].values
HZ = 50 # Sampling frequency in Hz.
duration = 30 # Total duration in seconds.
t = np.arange(len(data)) / HZ

# Extract the noise period (0 to 9 seconds).
noise_mask = (t >= 0) & (t < 9)
noise_part = data[noise_mask]
t_noise = t[noise_mask]

# Extract the stimulus period (12 to 30 seconds).
stim_mask = (t >= 12) & (t <= 30)
stim_part = data[stim_mask]
t_stim = t[stim_mask]
```

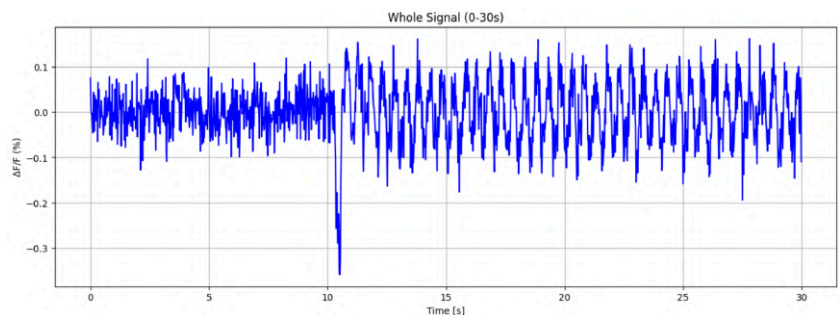
A. האות חולק לפי הדרישה לשלושת החלקים הבאים: הסיגנל כולו, 9 שניות ראשונות של רעש, שניות 12-30 התגובה לגירוי.

הסיגנל כולו:

```
plt.figure(figsize=(15, 5))

# Plot the entire signal.
plt.plot(t, data, color='blue')

# Add titles and labels.
plt.title('Whole Signal (0-30s)')
plt.xlabel('Time [s]')
plt.ylabel('ΔF/F (%)')
plt.grid(True)
```

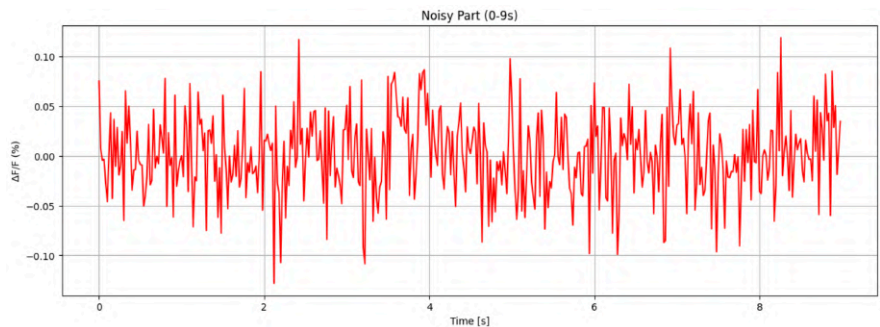


הרעש (שניות 0-9):

```
plt.figure(figsize=(15, 5))

# Plot the noise part of the signal.
plt.plot(t_noise, noise_part, color='red')

# Add titles and labels.
plt.title('Noisy Part (0-9s)')
plt.xlabel('Time [s]')
plt.ylabel('ΔF/F (%)')
plt.grid(True)
```



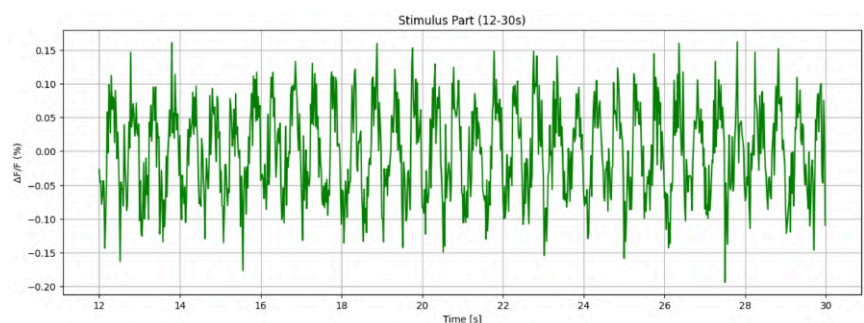
הגירוי (שניות 12-30):

```
plt.figure(figsize=(15, 5))

# Define and apply custom x-axis ticks from 12s to 30s in steps of 2s.
ticks = np.arange(12, 31, 2)
plt.xticks(ticks)

# Plot the stimulus part of the signal.
plt.plot(t_stim, stim_part, color='green')

# Add titles and labels.
plt.title('Stimulus Part (12-30s)')
plt.xlabel('Time [s]')
plt.ylabel('ΔF/F (%)')
plt.grid(True)
```



B. כעת, דגמנו מחדש בצורה ידנית, לפי התדרים המבוקשים. עבור כל תדר, יצרנו מערך חדש מתוך המידע על הגירוי, וכן מערך חדש של זמן שיתאים לקנה המידה של תדר הדגימה הרצוי.

:25 Hz

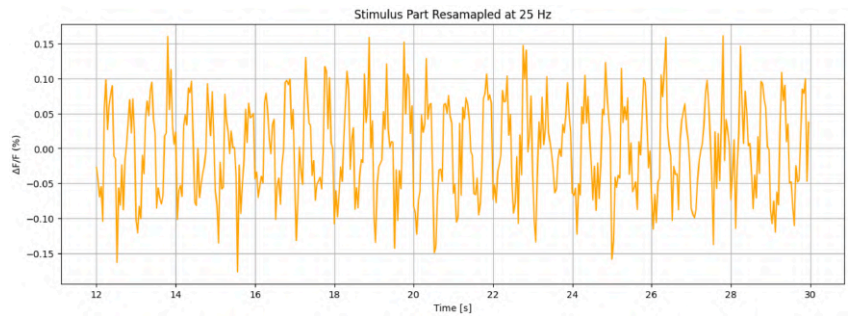
```
# Downsample the signal from 50 Hz to 25 Hz by taking every 2nd data point.
resampled_25 = stim_part[::2]

# Downsample the time vector as well.
t_25 = t_stim[::2]

# Create a new figure and apply custom x-axis ticks.
plt.figure(figsize=(15, 5))
plt.xticks(ticks)

# Plot the resampled 25 Hz signal.
plt.plot(t_25, resampled_25, color='orange')

# Add titles and labels.
plt.title('Stimulus Part Resampled at 25 Hz')
plt.xlabel('Time [s]')
plt.ylabel('ΔF/F (%)')
plt.grid(True)
```



:10 Hz

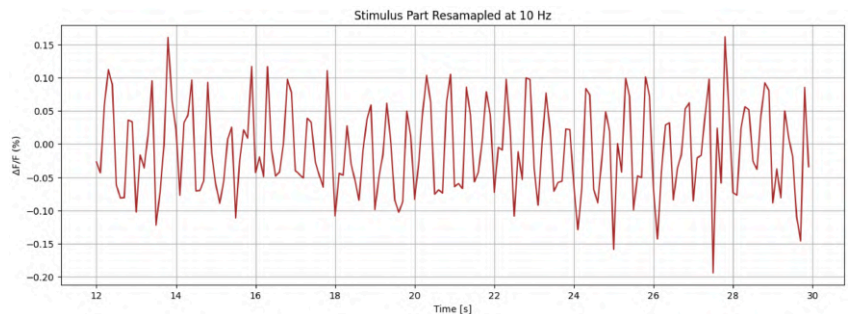
```
# Downsample the signal from 50 Hz to 10 Hz by taking every 5th data point.
resampled_10 = stim_part[::5]

# Downsample the time vector as well.
t_10 = t_stim[::5]

# Create a new figure and apply custom x-axis ticks.
plt.figure(figsize=(15, 5))
plt.xticks(ticks)

# Plot the resampled 10 Hz signal.
plt.plot(t_10, resampled_10, color='brown')

# Add titles and labels.
plt.title('Stimulus Part Resampled at 10 Hz')
plt.xlabel('Time [s]')
plt.ylabel('ΔF/F (%)')
plt.grid(True)
```



:3⅓ Hz

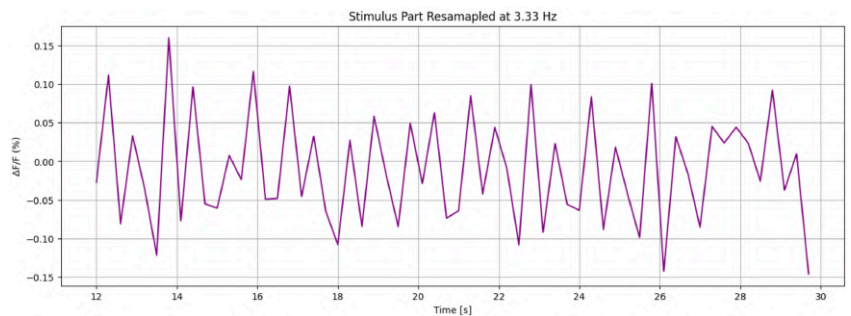
```
# Downsample the signal from 50 Hz to 3.33 Hz by taking every 15th data point.
resampled_3_33 = stim_part[::15]

# Downsample the time vector as well.
t_3_33 = t_stim[::15]

# Create a new figure and apply custom x-axis ticks.
plt.figure(figsize=(15, 5))
plt.xticks(ticks)

# Plot the resampled 3.33 Hz signal.
plt.plot(t_3_33, resampled_3_33, color='purple')

# Add titles and labels.
plt.title('Stimulus Part Resampled at 3.33 Hz')
plt.xlabel('Time [s]')
plt.ylabel('ΔF/F (%)')
plt.grid(True)
```



C. ראשית, ננתח כל תדר דגימה ונחשוב האם אנו צפויים לראות aliasing. לאחר מכן, נחשב את ה-aliasing עבור כל אחד

מהתדרים שבהם נצפה לראות זאת באמצעות הנוסחה $f_{alias} = \min_N |f - Nf_s|$, כאשר f הוא תדר האות, ו- f_s הוא תדר הדגימה.

לפי חוק Nyquist, עלינו לדגום בתדירות של לפחות פי 2 מתדר האות, (במקרה שלנו תדר האות הוא 2 Hz ולכן תדר הדגימה המינימלי יהיה 4 Hz) על מנת להימנע מאיבוד מידע.

עבור תדר דגימה של 25 Hz, וכן של 10 Hz, מדובר בתדר דגימה גבוה מ-4 Hz ולכן נוכל לדעת שלא יהיה aliasing ולא נאבד מידע.

עבור תדר דגימה של 3⅓ Hz, מדובר בתדר דגימה נמוך מ-4 Hz, ולכן נצפה לראות aliasing.

נחשב את התדר שנראה בפועל באמצעות הנוסחה שתיארנו, ונקבל: $f_{alias} = \left| 2 - 1 \cdot 3\frac{1}{3} \right| = 1\frac{1}{3}$

כלומר הגירוי ייראה בתדר של 1⅓ Hz.

כעת נספור את הפיקים באופן ידני עבור כל אחד מהתדרים.
 בגרפים מוצגות 18 שניות (משנייה 12 ועד 30).
 בתדירות של 25Hz וכן גם בתדירות של 10Hz, נראה שיש כ-36 פיקים ב-18 השניות, כלומר בממוצע 2 פיקים בשנייה,
 שזה בעצם 2Hz - ממש כמו בתדר הגירוי המקורי.
 בתדירות של 3.33Hz נבחין רק בכ-24 פיקים, ואכן 24 לחלק ל-18 ייתן לנו $1\frac{1}{3}$ פיקים בשנייה בממוצע.
 סה"כ קיבלנו כי אכן בכל התדירויות האלו ישנה התאמה בין הגרפים לבין חישוב האליאסינג התיאורטי.

D. כדי להבחין בשינוי של 0.1% בעוצמת הסיגנל, המערכת שלנו צריכה להיות בעלת רזולוציה שמאפשרת להפריד בין לפחות 1000 רמות עוצמה שונות (מפני ש-0.1% זה בעצם $1/1000$). לכן -

$$Bit = \log_2 \left(\frac{1}{0.001} \right) = \log_2 (1000) \approx 9.965 \Rightarrow Bit = 10$$

כלומר, נצטרך לפחות 10 ביטים על מנת להבחין בשינוי של 0.1%.

חלק 2: הפחתת רעש

1. כעת, נחשב את יחס האות לרעש SNR_{rms} בשתי הדרכים שנלמדו, על הדאטה בסעיף 1.A:

```
# Method 1: root of power ratio.
# Calculate the total power during the stimulus period (including noise).
power_signal = np.mean(stim_part**2)

# Calculate the power during the noise period.
power_noise = np.mean(noise_part**2)

# Pure stimulus power = signal power - noise power.
power_stim = power_signal - power_noise

# Calculate the square root of the power ratio.
snr_method_1 = np.sqrt(power_stim / power_noise)
print(f"SNR Method 1 (power ratio): {snr_method_1}")

# Method 2: mean of signal / STD of noise.
snr_method_2 = np.mean(stim_part) / np.std(noise_part)
print(f"SNR Method 2 (signal mean/noise STD): {snr_method_2}")

SNR Method 1 (power ratio): 1.3375628065932543
SNR Method 2 (signal mean/noise STD): -0.023089450715801853
```

בשיטה הראשונה קיבלנו -

$$SNR_{rms} = \sqrt{\frac{signal_{power} - noise_{power}}{noise_{power}}} \approx 1.337$$

ואילו בשיטה השנייה קיבלנו -

$$SNR_{rms} = \frac{\mu_{signal}}{\sigma_{noise}} \approx -0.023$$

ניתן לראות שיצאו 2 תוצאות שונות מאוד. השיטה השנייה הניבה תוצאה קרובה מאוד לאפס, מפני ששיטה זו מיועדת למדידת יחס אות לרעש של אותות קבועים, כמו למשל מתח חשמלי קבוע, ואילו באות שלנו יש אוסילציות של 2Hz - כלומר הוא אינו קבוע. לכן, חישוב ממוצע גורם לערכים החיוביים והשליליים (שנגרמים מתנודות האות) לבטל זה את זה ולהתאפס.

לעומת זאת, בשיטה הראשונה אנו קודם כל מעלים את כל הערכים בריבוע, ובכך "מעלימים" את הערכים השליליים, ככה שהם לא מתקזזים יותר עם הערכים החיוביים, ורק אז מחשבים ממוצע. כך בעצם ניתן למדוד את היחס האמיתי בין האות לרעש ללא קשר לסימן, ולכן קיבלנו SNR חיובי ואמין יותר למקרה שלנו.

2. ראשית, נציג את הקוד ואת תוצאות החישוב.

```
# Store the original SNR to calculate the improvement later.
origin_snr = snr_method_1
window_sizes = [2, 3, 5]

for w in window_sizes:
    # Truncate the stimulus array length to be divisible by current window size.
    stim_new_len = (len(stim_part) // w) * w
    stim_arr = stim_part[:stim_new_len]

    # Reshape into blocks of the window size and calculate the mean of each block.
    avg_stim = np.mean(stim_arr.reshape(-1, w), axis=1)

    # Repeat the same truncation and averaging process for the noise part.
    noise_new_len = (len(noise_part) // w) * w
    noise_arr = noise_part[:noise_new_len]
    avg_noise = np.mean(noise_arr.reshape(-1, w), axis=1)

    # Create the downsampled time vector.
    t_avg = t_stim[:stim_new_len:w]

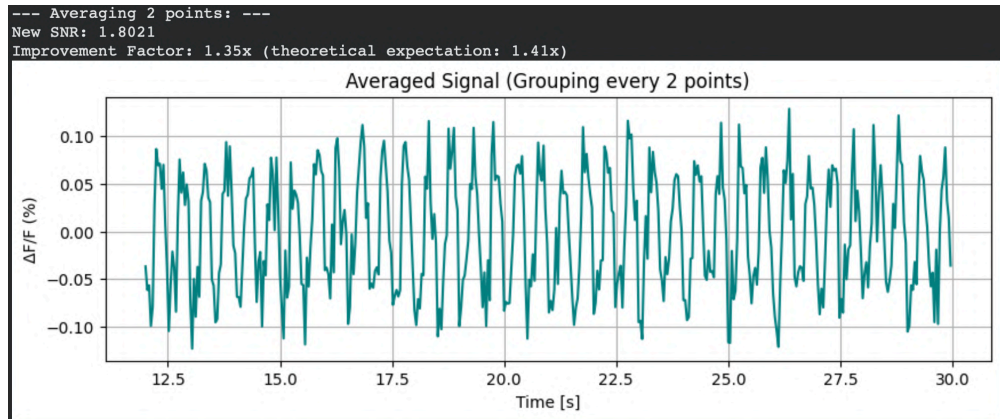
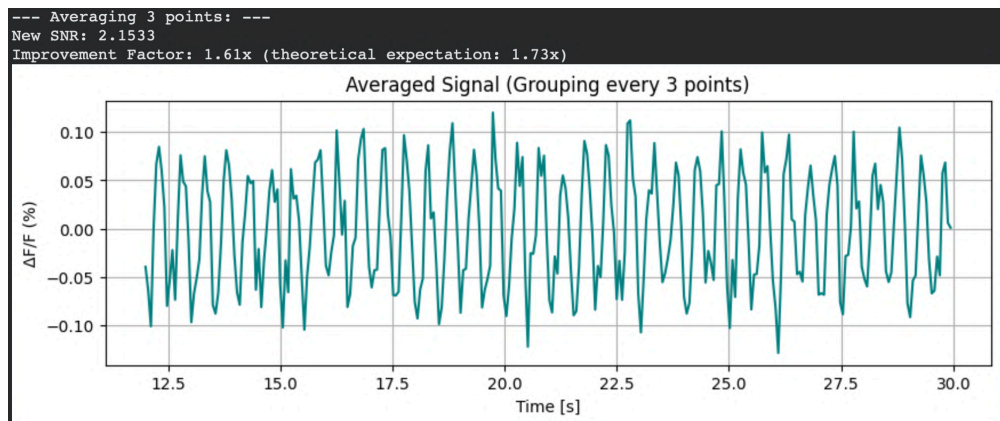
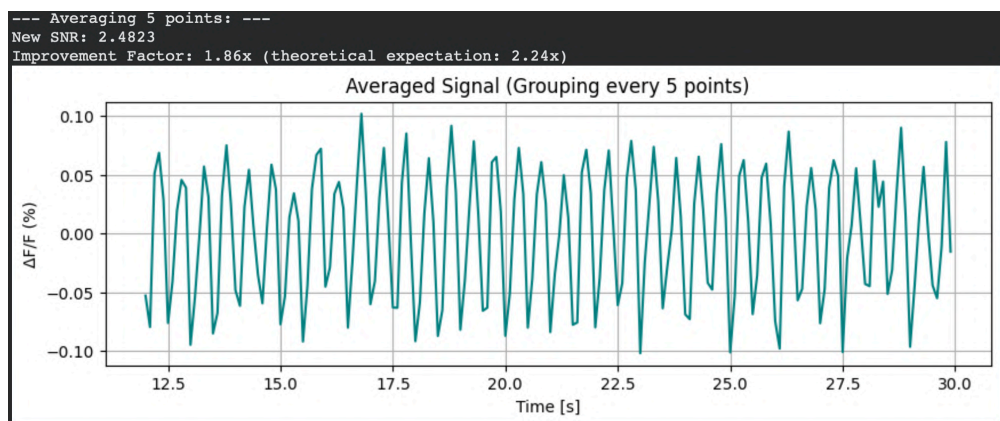
    # Calculate power for the values.
    signal_power = np.mean(avg_stim**2)
    noise_power = np.mean(avg_noise**2)

    # Calculate the new SNR and compare it to the original one.
    new_snr = np.sqrt((signal_power - noise_power) / noise_power)
    improvement = new_snr / origin_snr

    # According to theory, averaging N points improves SNR by sqrt(N).
    theoretical = np.sqrt(w)

    print(f"--- Averaging {w} points: ---")
    print(f"New SNR: {new_snr:.4f}")
    print(f"Improvement Factor: {improvement:.2f}x (theoretical expectation: {theoretical:.2f}x)")

    # Plot the averaged signal.
    plt.figure(figsize=(10, 3))
    plt.plot(t_avg, avg_stim, color='teal')
    plt.title(f'Averaged Signal (Grouping every {w} points)')
    plt.xlabel('Time [s]')
    plt.ylabel('ΔF/F (%)')
    plt.grid(True)
    plt.show()
```


מיצוע של 2 נקודות:מיצוע של 3 נקודות:מיצוע של 5 נקודות:

כפי שניתן לראות, ה-SNR משתפר בעקביות בכל שמגדילים את חלון המיצוע: שיפור של פי 1.35 עבור 2 נקודות, פי 1.61 עבור 3 נקודות, ופי 1.86 עבור 5 נקודות. השיפור הזה נובע מההבדל בין המאפיינים של האות לעומת המאפיינים של הרעש:

הרעש מכיל רכיבים מהירים ואקראיים שמתפלגים סביב ממוצע אפס, וכשאנו ממצעים או בעצם גורמים לערכי הרעש החיוביים והשליליים לקזז זה את זה. כתוצאה מכך, השונות וסטיית התקן של הרעש קטנים משמעותית. בנוסף, גל הגירוי שלנו (תדר של 2 Hz) הוא איטי ביחס לתדר הדגימה. מכיוון שתדר הדגימה הוא 50 Hz, מחזור שלם של הגל מורכב מ-25 דגימות. כאשר ממצעים חלונות קטנים (כפי שעשינו, 2-5 נקודות סמוכות), או בעצם ממצעים ערכים שקרובים מאוד זה לזה באמפליטודה. לכן, צורת הגל המקורית כמעט ואינה נפגעת.

3. מיצוע של 20 נקודות:

```

window = 20

# Truncate the stimulus array length to be divisible by window size.
stim_trunc_len = (len(stim_part) // window) * window
stim_truncated = stim_part[:stim_trunc_len]

# Reshape the data into blocks of 20 and calculate the mean for each block.
avg_stim_20 = np.mean(stim_truncated.reshape(-1, window), axis=1)

# Downsample the time vector.
t_avg_20 = t_stim[:stim_trunc_len:window]

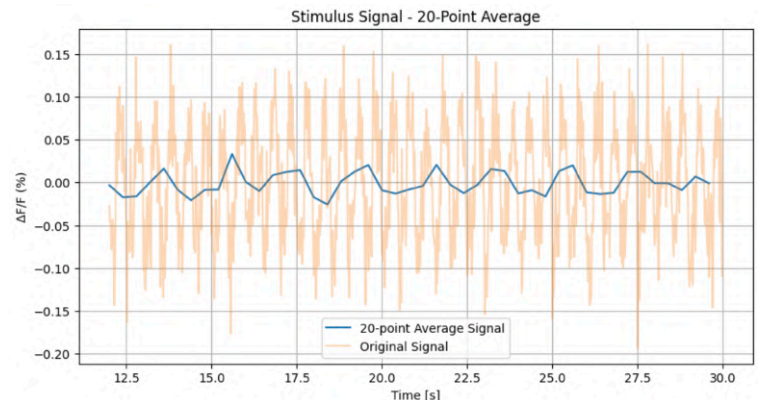
plt.figure(figsize=(10, 5))

# Plot the new averaged signal.
plt.plot(t_avg_20, avg_stim_20, label='20-point Average Signal')

# Plot the original signal in the background.
plt.plot(t_stim, stim_part, alpha=0.3, label='Original Signal')

# Add titles, labels and legend.
plt.title('Stimulus Signal - 20-Point Average')
plt.xlabel('Time [s]')
plt.ylabel('ΔF/F (%)')
plt.legend()
plt.grid(True)
plt.show()

```



ישנן שתי סיבות שבגללן איננו יכולים לראות עוד את האוסילציות המקוריות של הסיגנל: חלון מיצוע רחב מדי: תדר הדגימה המקורי הוא 50Hz, ולכן מחזור שלם של אות הגירוי (2Hz) מורכב מ-25 דגימות. שימוש בחלון מיצוע של 20 נקודות אומר שכל נקודה באות המעובד מייצגת ממוצע של 80% מאורך המחזור המקורי (20/25). בתוך חלון רחב כזה, נכללים חלקים רבים מהערכים החיוביים והשליליים של הגל, והם כמעט ומבטלים זה את זה, ולכן נוצרת הנחתה כה משמעותית של האמפליטודה המקורית. הפרת תנאי Nyquist: המיצוע מהווה בעצם דגימה מחדש של האות. עבור חלון של 20 נקודות, תדר הדגימה החדש הוא 2.5Hz (50/20). לפי תנאי Nyquist, תדר דגימה זה מאפשר לשחזר אותות בתדר מירבי של 1.25Hz בלבד. התדר המקורי שלנו גבוה מזה (2Hz) ולכן נוצר aliasing. התנודות הקטנות שרואים בגרף החדש מייצגות תדר אליאס (מדומה) של 0.5Hz, ולא משקף את הסיגנל המקורי.

4. החלקה באמצעות חלון מלבני בגודל 3, פעמיים על אותו החלון:

```

# Create a rectangular kernel (size 3).
kernel_size = 3
rect_kernel = np.ones(kernel_size) / kernel_size

# Apply the kernel twice to smooth the signal.
smooth_1 = np.convolve(stim_part, rect_kernel, mode='same')
smooth_2 = np.convolve(smooth_1, rect_kernel, mode='same')

```

5. החלקה באמצעות חלון משולש בגודל 5, פעם אחת:

```

# Create a triangular kernel (weighted moving average).
tri_weights = np.array([1, 2, 3, 2, 1])
tri_kernel = tri_weights / np.sum(tri_weights)

# Apply the kernel to the original signal.
smooth_tri = np.convolve(stim_part, tri_kernel, mode='same')

```

6. כעת נציג את הגרפים של ההחלקות שביצענו בסעיף 2.4 ובסעיף 2.5:

```

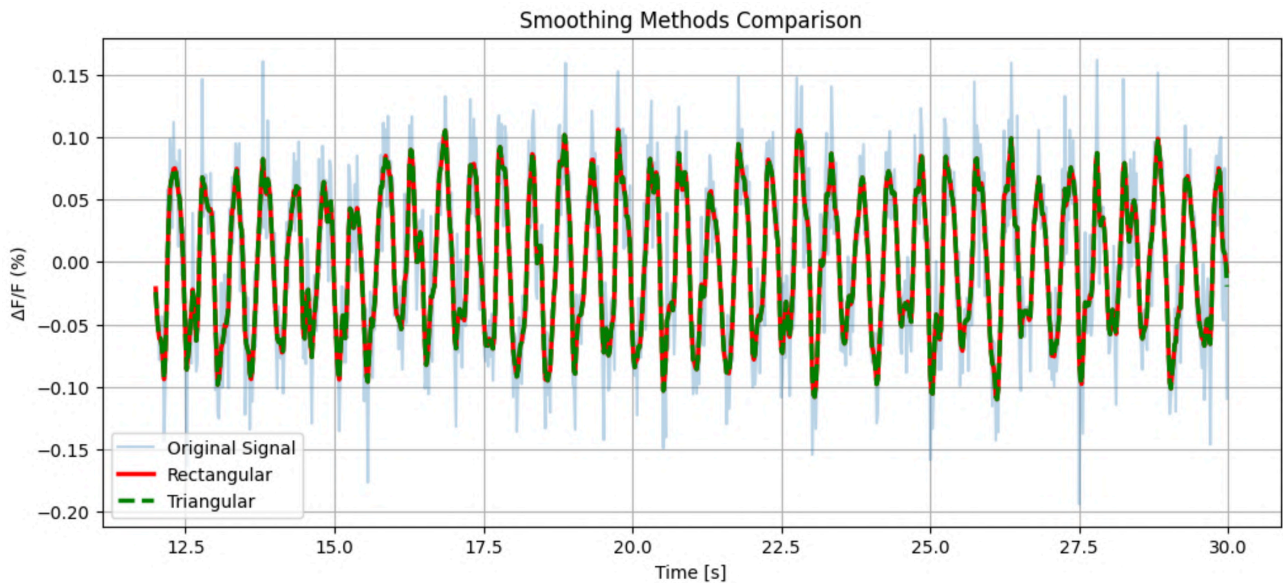
plt.figure(figsize=(12, 5))

# Plot the original signal in the background (transparent).
plt.plot(t_stim, stim_part, alpha=0.3, linewidth=1.5, label='Original Signal')

# Plot the smoothed signals (Triangular is dashed).
plt.plot(t_stim, smooth_2, color='red', linewidth=2.5, label='Rectangular')
plt.plot(t_stim, smooth_tri, color='green', linewidth=2.5, linestyle='--', label='Triangular')

plt.title('Smoothing Methods Comparison')
plt.xlabel('Time [s]')
plt.ylabel('ΔF/F (%)')
plt.legend()
plt.grid(True)
plt.show()

```



מהשוואת הגרפים ניתן לראות כי האות שעבר פעמיים החלקה מלבנית והאות שעבר פעם אחת החלקה משולשת זהים לחלוטין וחופפים זה לזה.

שתי השיטות בעצם הפחיתו את הרעש של התדרים הגבוהים בצורה משמעותית, ושפרו על תדר הגירוי המקורי. החפיפה הזו נוצרה מפני שקונבולוציה של חלון מלבני עם עצמו מייצרת חלון משולש, זאת אומרת- השיטות שקולות מבחינה מתמטית.

7. החלקה נעה (כלומר, באמצעות חלון נע) מתפקדת כ-Low-Pass Filter מפני שהיא מנחיתה תדרים גבוהים ומשמרת תדרים נמוכים. דבר זה מתאפשר בזכות האופי הפיזיקלי של התדרים.

תדרים גבוהים מתאפיינים בשינויים מהירים וחדים באמפליטודה מנקודת זמן אחת לשנייה (תנודות חיוביות ושליליות מהירות סביב קו האפס). כאשר מפעילים חלון מיצוע על נקודות סמוכות, תנודות מנוגדות אלו מקזזות זו את זו בתוך החלון, וכתוצאה מכך האמפליטודה של התדרים הגבוהים קטנה משמעותית (בעצם נשאפת לאפס). תדרים נמוכים (כמו אות הגירוי שלנו) משתנים לאט ביחס לתדר הדגימה. כלומר, נקודות סמוכות בתוך החלון הן בעלות ערכים כמעט זהים, ומיצוע על ערכים דומים שומר על האמפליטודה המקורית שלהם.

כך, פעולת ההחלקה בעצם "ניקתה" את הרעשים בתדרים הגבוהים, והשאירה בשלמותן את האוסילציות של האות המקורי.

חלק 3.1: ניתוח ספייקים

A. קצב הירי הממוצע של כל ניורון חושב על ידי סכימת כל הספייקים שלו ב-100 הטריאלים, וחלוקתם בזמן ההקלטה הכולל. להלן הקוד והתוצאות:

```
# Calculate total spikes per trial by summing all columns.
spike_count = df_spikes.drop(columns=['trial_neuron']).sum(axis=1)

# Convert total spike counts to firing rate (Hz).
firing_rates = spike_count / duration # duration = 30 [s].

# Extract the neuron letter from the string format.
df_spikes['neuron'] = df_spikes['trial_neuron'].apply(lambda x: str(x).split('_')[1])

# Add the firing rates to the array and calculate the average per neuron.
df_spikes['firing_rate'] = firing_rates
mean_rates = df_spikes.groupby('neuron')['firing_rate'].mean()

print("--- Mean Firing Rates (Hz) ---")
print(mean_rates)
```

```
--- Mean Firing Rates (Hz) ---
neuron
A    11.670333
B    42.550000
C   100.000000
D    49.984333
E    28.679000
```

B. בסעיף זה חושב ערך ה-Fano Factor עבור כל ניורון עם חלון של שנייה אחת. להלן הקוד והתוצאות:

```
# Extract the first 1 second columns (1000 ms).
window_data = df_spikes.iloc[:, 1:1001]

# Count the total number of spikes in this window for each trial.
df_spikes['counts_1sec'] = window_data.sum(axis=1)

# Calculate variance and mean of the spike counts for each neuron.
results = df_spikes.groupby('neuron')['counts_1sec'].agg(['var', 'mean'])

# Fano Factor = Variance / Mean.
results['Fano_Factor'] = results['var'] / results['mean']

print("--- Fano Factor for the first window ---")
print(results['Fano_Factor'])
```

```
--- Fano Factor for the first window ---
neuron
A    0.986133
B    0.574943
C    0.000000
D    0.010218
E    6.653800
```

C. כפי שניתן לראות, ערך ה-Fano Factor של ניורון A קרוב מאוד ל-1. משמעות הדבר היא שהשונות של הספייקים שלו כמעט שווה לממוצע שלהם, ממש כפי שקורה בתהליך פואסוני. כלומר- ניורון A הוא ניורון פואסוני. תכונה זו בעצם אומרת שהניורון יורה ללא תלות בזמן שעבר מאז הירי הקודם, הירי שלו אקראי.

D. שני הגרפים מציגים התפלגות מעריכים של מרווחי הזמן בין ספייקים. ראשית, נראה את הקוד (משמאל לימין) ואת הגרפים:

```
# Sampling frequency.
HZ = 1000

# Remove unnecessary columns to isolate the binary spike data.
drop_cols = ['trial_neuron', 'neuron', 'firing_rate', 'counts_1sec']
spike_data = df_spikes.drop(columns=drop_cols)

# Extract spike arrays for neurons A and B.
trials_A = spike_data[df_spikes['neuron'] == 'A'].values
trials_B = spike_data[df_spikes['neuron'] == 'B'].values

# Create a figure with 1 row and 2 columns to plot side-by-side.
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Function to calculate and plot ISI.
def plot_ISI(df, ax, title):
    distances = []
    bin_size = 1 / HZ # Size of one time bin in seconds (0.001s).

    for trial in df:
        # Find the timestamps where spikes occurred.
        spikes_index = [i for i, val in enumerate(trial) if val]

        # Calculate the time differences between consecutive spikes.
        dis = np.diff(spikes_index)
        dis_in_secs = dis * bin_size # Convert ms to s.
```

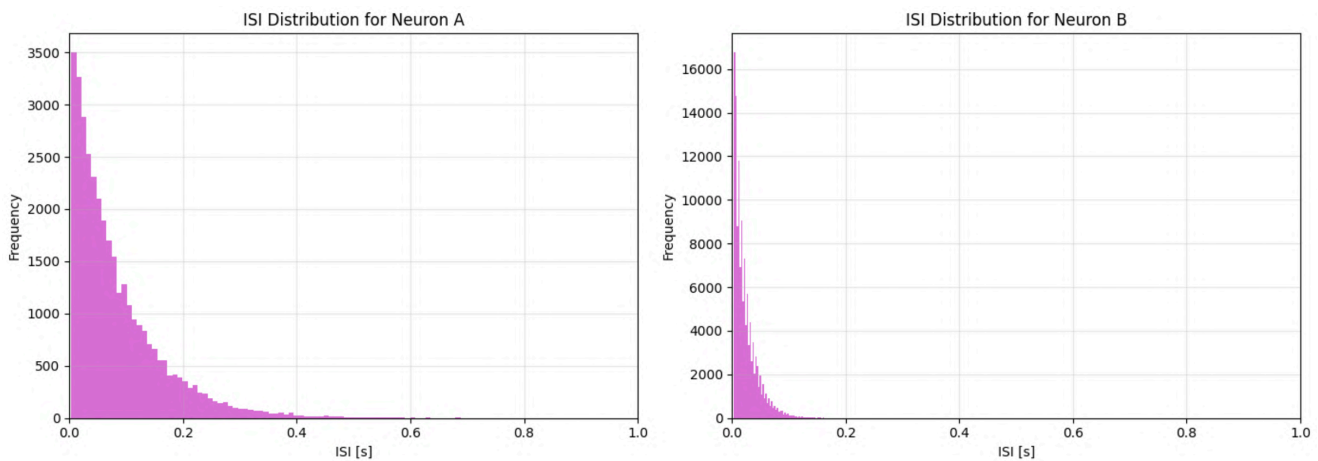
```
# Add the trial's intervals to the main list.
distances.extend(dis_in_secs)

# Plot the histogram using 100 bins for high resolution.
ax.hist(distances, bins=100, color='orchid')

# Add titles and labels for the specific subplot.
ax.set_title(title)
ax.set_xlabel('ISI [s]')
ax.set_ylabel('Frequency')
ax.set_xlim(0, 1)
ax.grid(True, alpha=0.3)

# Call the function for both neurons.
plot_ISI(trials_A, axes[0], 'ISI Distribution for Neuron A')
plot_ISI(trials_B, axes[1], 'ISI Distribution for Neuron B')

plt.tight_layout()
plt.show()
```

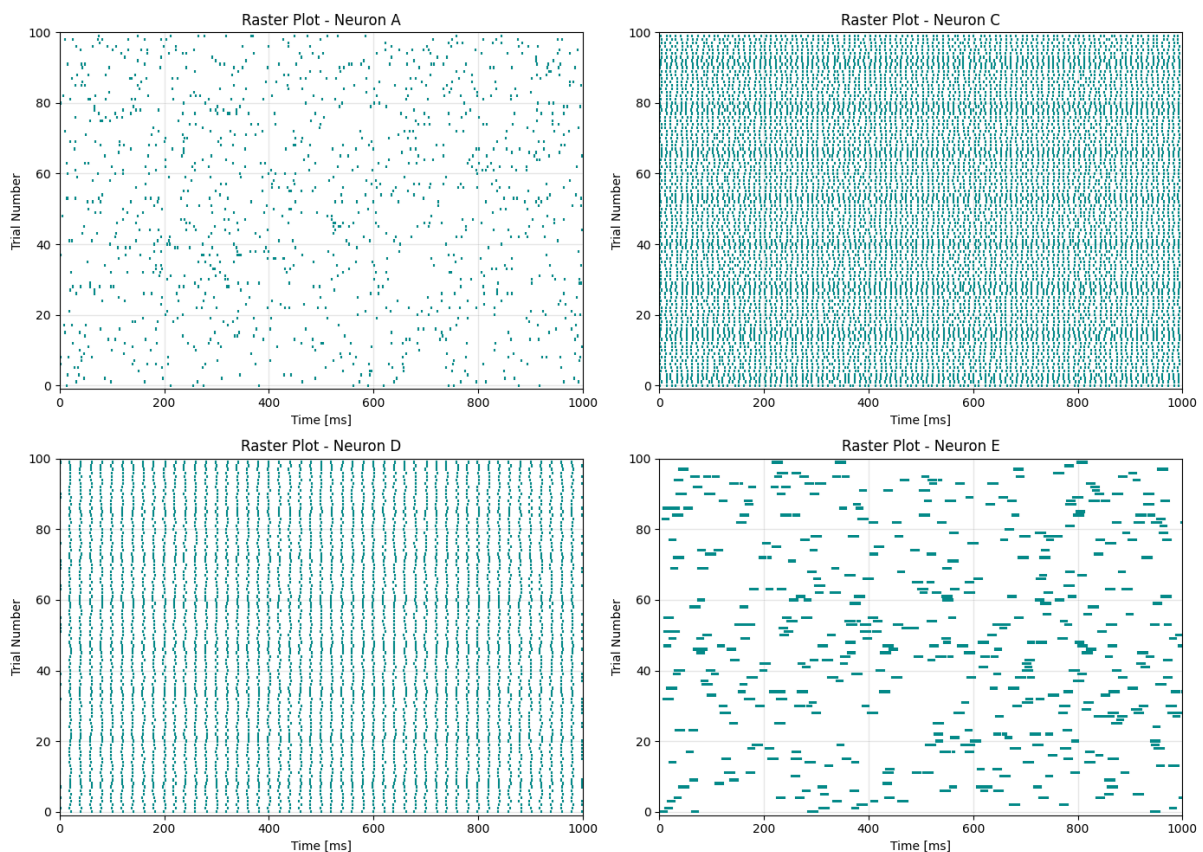
כעת, ננתח את הגרפים עבור כל ניירון.

ניירון A - בגרף ניתן לראות דעיכה מעריכית. העמודה הגבוהה ביותר נמצאת משמאל, וככל שמתקדמים עם הזמן התדירות הולכת ויורדת בצורה חלקה יחסית. זו בעצם צורת התפלגות שמתאימה לירי של ניירון פואסוני, ולכן זה מתיישב עם התוצאה של FF על ניירון זה.

ניירון B - גם כאן ניתן לראות דעיכה מעריכית, אך הרבה יותר חדה מאשר בגרף של ניירון A. הגרף כולו צפוף יותר בציוד השמאלי. כלומר, ניירון זה יורה בצורה עקבית ומהירה, ואכן התוצאה של FF על ניירון B הצביעה לנו על ירי יותר סדיר מאשר ניירון A.

ההבדל בפיזור נובע מקצב הירי של שני הניירונים. ניירון B יורה בתדירות גבוהה משמעותית מניירון A. כשניירון יורה יותר באותו חלון זמן, ה-ISI (ההפרש בין מרווחי הירי) חייב להיות קצר יותר. דבר זה גם מתבטא בזנב- בגרף של ניירון B הזנב קצר משמעותית מבגרף של ניירון A.

E. בסעיף זה יצרנו גרף רסטר, אשר מציג את זמני הירי של ניירונים A, C, D, E לאורך כל הטריאלים במהלך השנייה הראשונה של ההקלטה.



```
# Define the neurons to plot.
neurons_to_plot = ['A', 'C', 'D', 'E']

# Create a 2x2 grid of subplots, and flatten the matrix into 1D array to iterate easily.
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for i, neuron in enumerate(neurons_to_plot):

    # Filter data for the specific neuron, and extract only the first 1000 ms.
    neuron_df = spike_data[df_spikes['neuron'] == neuron]
    first_sec_data = neuron_df.iloc[:, :1000]

    # Find the time indices where spikes occurred for each trial.
    spike_positions = [np.where(row == 1)[0] for _, row in first_sec_data.iterrows()]

    # Plot the spikes as vertical lines.
    axes[i].eventplot(spike_positions, linelengths=0.8, colors='darkcyan')

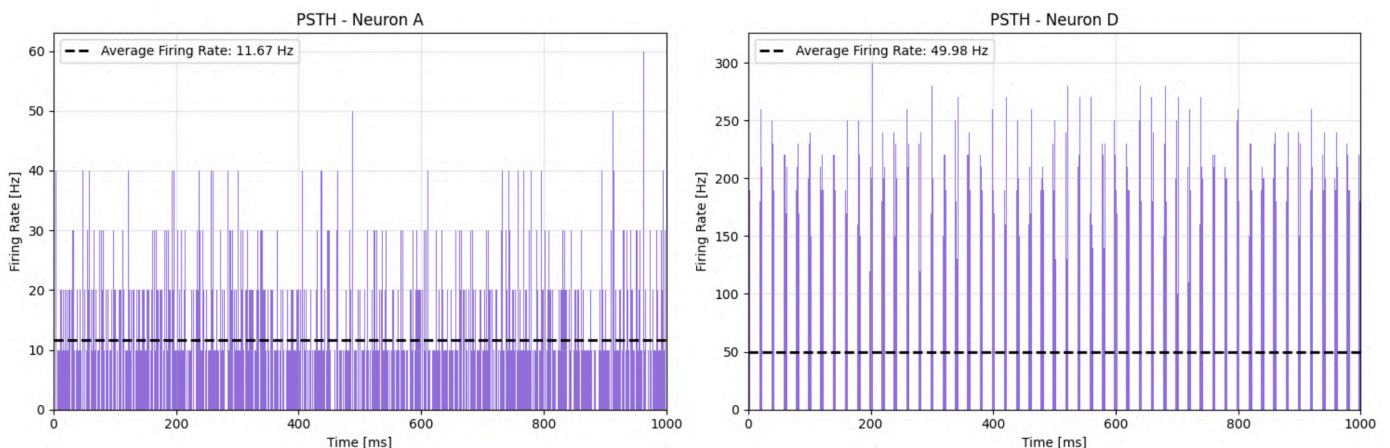
    # Add title, labels, and set axis limits.
    axes[i].set_title(f'Raster Plot - Neuron {neuron}')
    axes[i].set_xlabel('Time [ms]')
    axes[i].set_ylabel('Trial Number')

    axes[i].set_xlim(0, 1000)
    axes[i].set_ylim(-1, len(spike_positions))
    axes[i].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```

F. נוירון E הוא נוירון מתפרץ. בגרף ה-raster של נוירון זה ניתן לזהות תבנית ירי שאינה אחידה. במקום פיזור אקראי של ספייקים לאורך ציר הזמן (נוירון פואסוני) או הופעה במרווחים קבועים (נוירון סדיר), הספייקים כאן מופיעים כ-clusters צפופים מאוד. בין הצבירים האלו ניתן להבחין בפרקי זמן שקטים שבהם הנוירון כמעט ואינו יורה. ניזכר בערך ה-Fano Factor של נוירון E: 6.653800 - ערך הגדול משמעותית מ-1. מדד ה-FF בודק את היחס בין השונות לממוצע. מפני שהנוירון הזה עובר בין שקט מוחלט לירי מסיבי, כמות הספייקים מחלון זמן אחד לאחר משתנה בצורה דרסטית, כלומר- השונות מאוד גבוהה. לכן, נוירון E מוגדר סטטיסטית ותצפיתית כנוירון מתפרץ.

G. בסעיף זה נציג את ה-PSTH של השנייה הראשונה של ההקלטה, עבור נוירונים A, D.



ניתן לראות שהפיקים של נוירון D גבוהים יותר מקצב הירי הממוצע שלו. הפער נובע מההבדל בין קצב ירי **ממוצע** לבין קצב ירי **רגעי**.

קצב הירי הממוצע מחושב על פני פרק זמן ארוך (כל אורך ההקלטה) וכולל בתוכו גם את תקופות השקט של הנוירון, מה שמושך את הערך הכולל של הממוצע כלפי מטה.

לעומת זאת, קצב הירי הרגעי (PSTH) מודד את הפעילות בחלונות זמן זעירים (במקרה שלנו- 1 מילי-שנייה). מכיוון שנוירון D מסונכרן ומתוזמן מאוד בין הטריאלים (ניתן להסיק זאת מכך שערך ה-FF שלו כמעט 0), כמות גדולה של ספייקים מתרכזת בדיוק באותו חלון זמן. כאשר אנו סוכמים את הספייקים ב-PSTH בנקודות הזמן שבהן הנוירון יורה, אנו מקבלים ריכוז גבוה מאוד של ספייקים מכל הטריאלים באותו חלון. לכן, קצב הירי הרגעי שקיבלנו הוא גבוה מאוד.

```

# Define experimental parameters.
num_trials = 100
bin_width_sec = 1 / HZ    # 1 ms bin width in seconds.

# Create a figure with 1 row and 2 columns for side-by-side plotting.
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
axes = axes.flatten()

for i, neuron in enumerate(['A', 'D']):
    # Extract all trials for the current neuron.
    neuron_trials = spike_data[df_spikes['neuron'] == neuron].values

    # Slice only the first 1 second (1000 ms) of data.
    first_sec_data = neuron_trials[:, :HZ]
    hist_data = []

    # Collect all spike timestamps (in ms) across all trials for the first second.
    for trial in first_sec_data:
        spike_pos = np.where(trial == 1)[0]
        hist_data += list(spike_pos)

    # Calculate the overall average firing rate across the entire recording.
    total_spikes = np.sum(neuron_trials)
    avg_fr = total_spikes / (num_trials * duration)

    # Create weights to normalize the histogram Y-axis into Firing Rate (Hz).
    weights = np.ones_like(hist_data) / (num_trials * bin_width_sec)

    # Plot the PSTH.
    axes[i].hist(hist_data, bins=HZ, weights=weights, width=1.5, color='mediumpurple')

    # Add a line representing the overall average firing rate.
    axes[i].axhline(avg_fr, color='black', linestyle='--', linewidth=2, label=f'Average Firing Rate: {avg_fr:.2f} Hz')

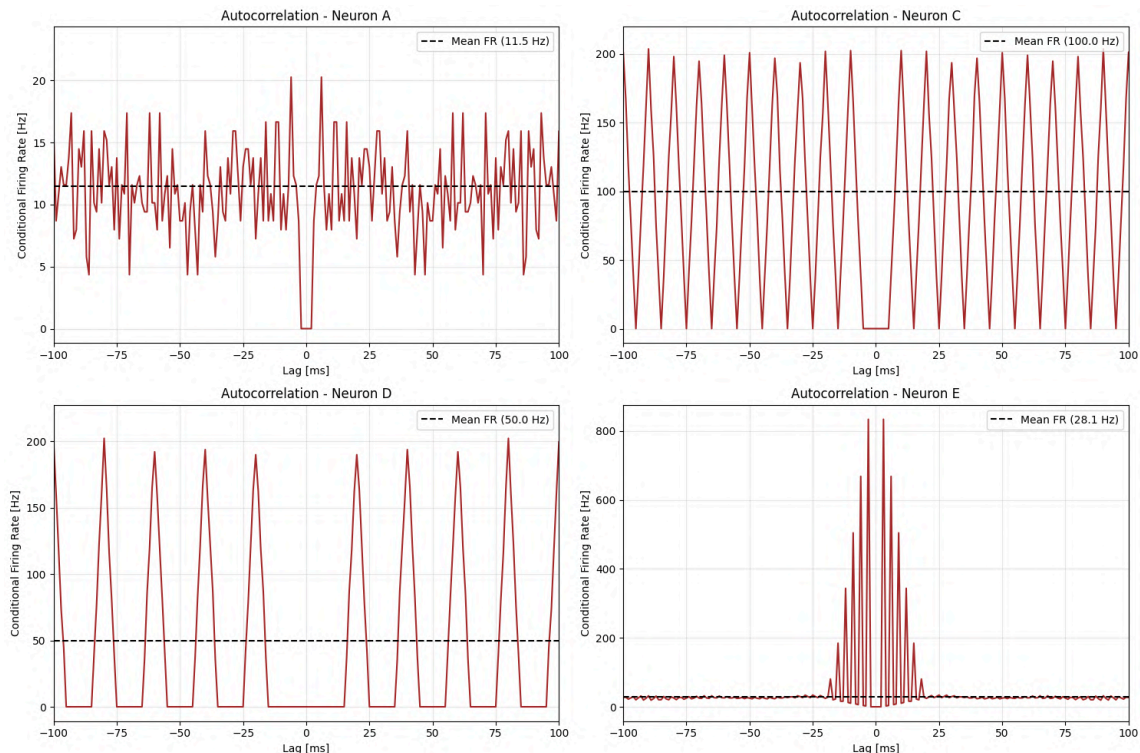
    # Add titles and labels.
    axes[i].set_title(f'PSTH - Neuron {neuron}')
    axes[i].set_xlabel('Time [ms]')
    axes[i].set_ylabel('Firing Rate [Hz]')
    axes[i].legend(loc='upper left')
    axes[i].set_xlim(0, 1000)
    axes[i].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

חלק 3.2: ניתוח קורלציה

A. בגרפים בסעיף זה ניתן לראות את החישוב האוטו-קורלציה עבור הניורונים המבוקשים, ואת התקופה הרפרקטורית של כל אחד מהם ניתן לראות סביב lag 0. באיזור זה נזהה צניחה חדה של הגרף לקו האפס למשך מספר מילי-שניות. משך הזמן הזה משתנה בין הניורונים, ומתאר את פרק הזמן שבו ההסתברות לירי היא אפסית.



```
# Define neurons to analyze and calculate the number of samples in 2 minutes (120s).
neurons_to_plot = ['A', 'C', 'D', 'E']
two_mins_samples = 120 * HZ

# Create a 2x2 grid of subplots and flatten for easy iteration.
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.flatten()

for i, neuron in enumerate(neurons_to_plot):
    # Extract all trials for the specific neuron.
    neuron_df = spike_data[df_spikes['neuron'] == neuron].values

    # Flatten all trials into one continuous recording and keep only the first 2 minutes.
    flattened = neuron_df.flatten()
    data_2_mins = flattened[:two_mins_samples]

    # Count total spikes in this window.
    total_spikes = np.sum(data_2_mins)

    # Calculate autocorrelation efficiently using FFT.
    auto_corr = signal.correlate(data_2_mins, data_2_mins, mode='full', method='fft')

    # Generate the corresponding lag axis.
    lags = signal.correlation_lags(len(data_2_mins), len(data_2_mins), mode='full')

    # Normalize raw coincidences to firing rate.
    normalized = (auto_corr / total_spikes) * HZ
```

```
# Manually zero out lag 0 to avoid distorting the plot's scale.
center_idx = np.where(lags == 0)[0][0]
normalized[center_idx] = 0

# Calculate the overall mean firing rate.
mean_fr = total_spikes / 120

# Plot the autocorrelation function.
axes[i].plot(lags, normalized, color='brown')
axes[i].set_title(f'Autocorrelation - Neuron {neuron}')
axes[i].set_xlabel('Lag [ms]')
axes[i].set_ylabel('Conditional Firing Rate [Hz]')

# Add the mean firing rate reference line and a legend.
axes[i].axhline(mean_fr, color='black', linestyle='--', label=f'Mean FR ({mean_fr:.1f} Hz)')
axes[i].legend(loc='upper right')

# Zoom in on the central window (-100 ms to +100 ms).
axes[i].set_xlim(-100, 100)
axes[i].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

B. ניורון C הוא ניורון סדיה. בגרף האוטו-קורלציה שלו ניתן לראות פיקים גבוהים וסימטריים במרווחי זמן קבועים, כלומר מדובר בתבנית מחזורית מובהקת. התנהגות זו תואמת את מדד ה-FF של ניורון C (שהוא 0.000000). מכיוון שניורון זה יורה בקצב קבוע, כמות הספייקים בכל חלון נשארת זהה בין טריאלים. כתוצאה מכך, בכל חלון זמן השונות היא אפסית, מה שמוביל לערך FF אפסי (שכן מדובר בשונות חלקי הממוצע עבור כל חלון זמן).

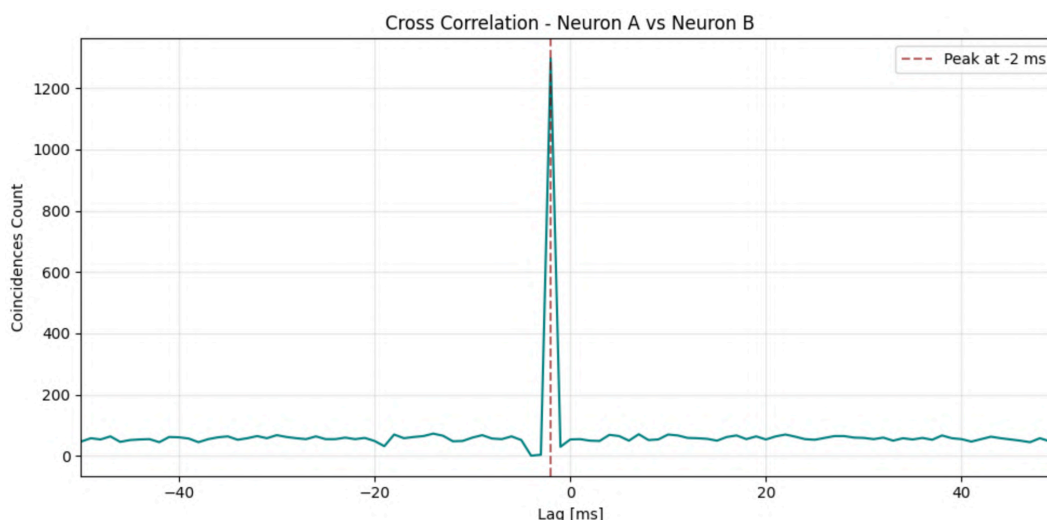
C. לפי האוטו-קורלציה ניתן לראות שגם ניורון D הוא ניורון סדיה, אך הוא יורה בקצב נמוך יותר מאשר ניורון C. דבר זה בא לידי ביטוי בגרפים על ידי מרווחי הזמן בין הפיקים, שעבור ניורון D הם ארוכים יותר. בניורון C הפיקים צפופים יותר- דבר המעיד על תדר ירי גבוה יותר, ואילו בניורון D הפיקים מרווחים יותר- דבר המעיד על

תדר ירי נמוך יותר. בנוסף, ניתן לראות את ההבדל הנ"ל בערך קצב הירי הממוצע. קצב הירי הממוצע של ניורון C הוא 100Hz ואילו של ניורון D הוא 50Hz - בדיוק חצי משל C.

D. בניורון E סביב lag 0 ישנה "גבעה" רחבה וגבוהה של הסתברות ירי, וככל שמתרחקים מ-lag 0 ה"גבעה" דועכת. צורה זו אופיינית לניורון מתפרץ - היא מעידה על כך שאם הניורון ירה ספייק בזמן נתון, קיימת הסתברות גבוהה מאוד שהוא ירה ספייקים נוספים ברצף מהיר מיד לאחר מכן, כאשר בין הספייקים יש תקופות שקטות קצרות.

1. מספר הפיקים מצביע על המספר הממוצע של ספייקים המרכיבים התפרצות בודדת.
2. משמעות המינימום המקורי אחרי כל פיק היא בעצם התקופה הרפרקטורית. זהו פרק זמן אופייני בין ספייקים בתוך אותה התפרצות, בו הניורון לא יכול לירות שוב לאחר הירי הקודם, למרות שהוא עדיין בהתפרצות.

E. גרף זה מתאר את הקרוס-קורלציה בין הניורונים A, B בשתי הדקות הראשונות, ללא נרמול.



```
# Extract and flatten trials for neuron A, keeping only the first 2 minutes.
trials_A = spike_data[df_spikes['neuron'] == 'A'].values
data_A = trials_A.flatten()[:two_mins_samples]

# Repeat for neuron B.
trials_B = spike_data[df_spikes['neuron'] == 'B'].values
data_B = trials_B.flatten()[:two_mins_samples]

# Calculate cross-correlation using FFT for efficiency.
cross_corr = signal.correlate(data_A, data_B, mode='full', method='fft')
lags = signal.correlation_lags(len(data_A), len(data_B), mode='full')

# Find the lag (in ms) where the peak occurs.
peak_idx = np.argmax(cross_corr)
peak_lag = lags[peak_idx]
```

```
# Plot the cross-correlation result.
plt.figure(figsize=(10, 5))
plt.plot(lags, cross_corr, color='teal')
plt.title('Cross Correlation - Neuron A vs Neuron B')
plt.xlabel('Lag [ms]')
plt.ylabel('Coincidences Count')

# Add a vertical dashed line to highlight the exact lag of the peak.
plt.axvline(x=peak_lag, color='darkred', linestyle='--', alpha=0.6, label=f'Peak at {peak_lag} ms')
plt.legend()

# Zoom in on a narrow window.
plt.xlim(-50, 50)
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

F.

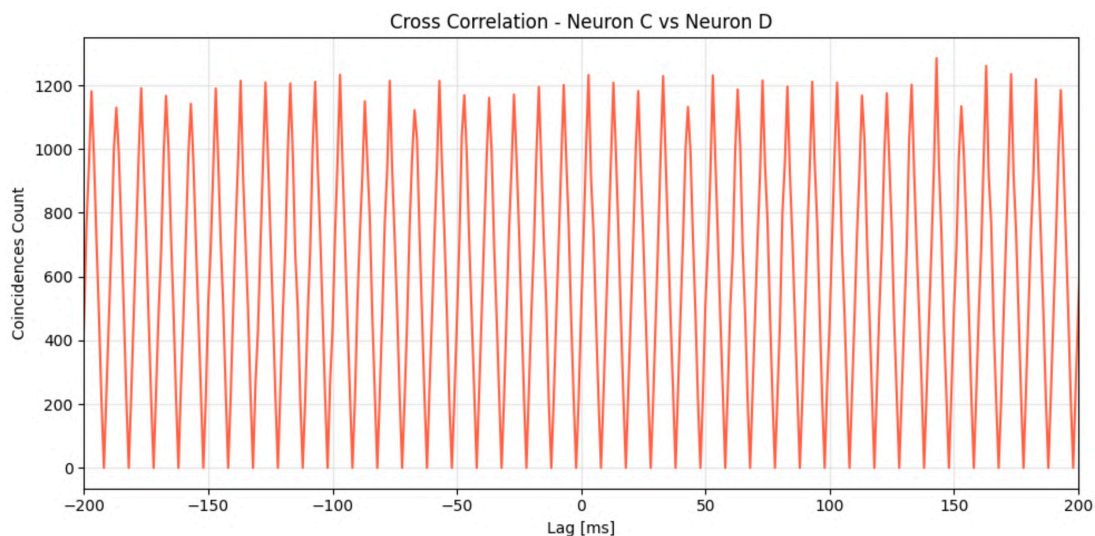
1. לפי גרף הקרוס-קורלציה, ניתן לקבוע כי יש קשר מובהק בין הניורונים. הפיק החד שמופיע בגרף מעיד על תלות בין זמני הירי של שני הניורונים.

- הקשר הוא מעורר. ניתן לקבוע זאת מהפיק החיובי שנוצר בגרף, המעיד על כך שיש עלייה חדה בכמות הפעמים שהניורונים ירו בסמיכות. קשר מעכב היה מתבטא בירידה מתחת לקו הבסיס.
- זמן ההשהיה בין הניורונים הוא 2 מילי-שניות.
- ניורון A מעורר את ניורון B. נבחין כי הפיק בגרף נמצא ב-lag שלילי. כאשר האות הראשון בחישוב הוא A והשני הוא B, משמעות הדבר היא שכדי לקבל את הפיק היינו צריכים להזיז את B 2 מילי-שניות "אחורה" כדי לקבל חפיפה עם הירי של ניורון A. כלומר, במציאות ניורון A יורה קודם, ו-2ms אחריו יורה ניורון B.

2. בהנחה שניורון A מעורר את ניורון B בכל פעם שהוא יורה, האמפליטודה של הפיק מייצגת את המספר הכולל של הפעמים שניורון A ירה במהלך 2 הדקות שניתחנו. במקרה זה, האמפליטודה עומדת על כ-1300. על מנת לחשב את קצב הירי הממוצע של ניורון A בשניות, נצטרך לחלק את כמות הספייקים בזמן שנמדד (בשניות). כלומר $1300/120$, ונקבל שקצב הירי הוא בערך 10.83Hz.

3. אם נוירון A מעורר את B בסבירות של 50%, אזי רק חצי מהספייקים של נוירון A גורמים לנוירון B לירות, ולכן כמות הפעמים בהן הם יירו ברצף תרד גם היא בדיוק בחצי. כלומר, האמפליטודה של הפיק תקטן ב-50% - במקום שתהיה סביב 1300, ערכה יהיה סביב 650. יחד עם זאת, מיקום הפיק על ציר הזמן יישאר קובע, שכן זמן ההולכה לא משתנה, וכן הרוחב שלו לא ישתנה.

G. גרף הקרוס-קורלציה בין נוירון C לנוירון D מציג תבנית מחזורית, רציפה ואוסילטורית, ללא פיק מרכזי בולט. ניתן להסיק מכך כי בין שני נוירונים אלו אין קשר ישיר כלשהו. אף אחד מהם לא מעורר או מעכב את השני. כפי שראינו בסעיפים הקודמים, שני הנוירונים הם סדירים, כך שקצב הירי של נוירון C גדול פי 2 מזה של D. ואכן, כאשר מחשבים קרוס-קורלציה בין שני אותות מחזוריים ובלתי-תלויים, התוצאה המתקבלת היא גל מחזורי. נשים לב כי הפיקים הרבים בגרף זה מייצגים את נקודות הזמן שבהן שני הנוירונים הללו מסתנכרנים. כאשר נוירון D יורה פעם אחת, נוירון C יורה פעמיים באותו חלון זמן בדיוק, ולכן נקודות הסנכרון ביניהם הן קבועות ומחזוריות. כלומר, תבנית זו אינה מעידה בהכרח על קשר סינפטי ישיר, אלא על סנכרון פאזה מסוים, למשל כתוצאה מקלט משותף שמכתיב לשניהם את הקצב.



```
# Extract and flatten trials for neuron A, keeping only the first 2 minutes.
trials_C = spike_data[df_spikes['neuron'] == 'C'].values
data_C = trials_C.flatten()[:two_mins_samples]

# Repeating for neuron D.
trials_D = spike_data[df_spikes['neuron'] == 'D'].values
data_D = trials_D.flatten()[:two_mins_samples]

# Calculate cross-correlation using FFT for efficiency.
cross_corr = signal.correlate(data_C, data_D, mode='full', method='fft')
lags = signal.correlation_lags(len(data_C), len(data_D), mode='full')

# Plot the cross-correlation result.
plt.figure(figsize=(10, 5))
plt.plot(lags, cross_corr, color='tomato')
plt.title('Cross Correlation - Neuron C vs Neuron D')
plt.xlabel('Lag [ms]')
plt.ylabel('Coincidences Count')

# Zoom in on a wider window to observe the oscillatory pattern.
plt.xlim(-200, 200)
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```